

Couche Liaison de données

Pr. Fatiha Djemili Tolba
Fatiha.djemili@univ-annaba.dz



Contenu du cours

- ✓ Fonctions
- ✓ les services possibles,
- ✓ Protocole HDLC (High-level Data Link Control).
- ✓ le contrôle de flux et
- ✓ le contrôle des erreurs (Techniques de détection d'erreur).

Couche 2 : fonctions

Assurer une communication fiable, c-à-d les données échangées par la couche réseau doivent être : dans l'ordre, sans erreur, sans perte et sans duplication.

- Encapsulation des paquets niveau 3 en trames niveau 2.
- Délimitation des trames et adressage de niveau 2.
- Etablissement/libération de la liaison.
- Fonction de séquençement.
- Contrôle de flux.
- Contrôle d'erreur.

Couche 2 : services possibles

1. Service *sans connexion, sans accuse de réception* : souvent offert quand le taux d'erreur faible, notamment dans les réseaux locaux (ex : Ethernet en modèle TCP/IP).
2. Service *avec connexion avec accuse de réception*: les paquets seront reçus dans l'ordre d'émission et une seule fois (ex : HDLC en modèle OSI).
3. Service *sans connexion avec accuse de réception (optionnel)* : plutôt offert quand la liaison est peu fiable (ex : protocole send and wait).

Couche 2 : HDLC

Généralité (HIGHT-level DATA LINK CONTROL)

- Est un protocole point-à-point de référence normalisé par l'ISO. Offre un service de transfert de données fiable et efficace entre deux systèmes synchrones utilisant le mode connecté. L'unité de transfert d'HDLC est la trame.

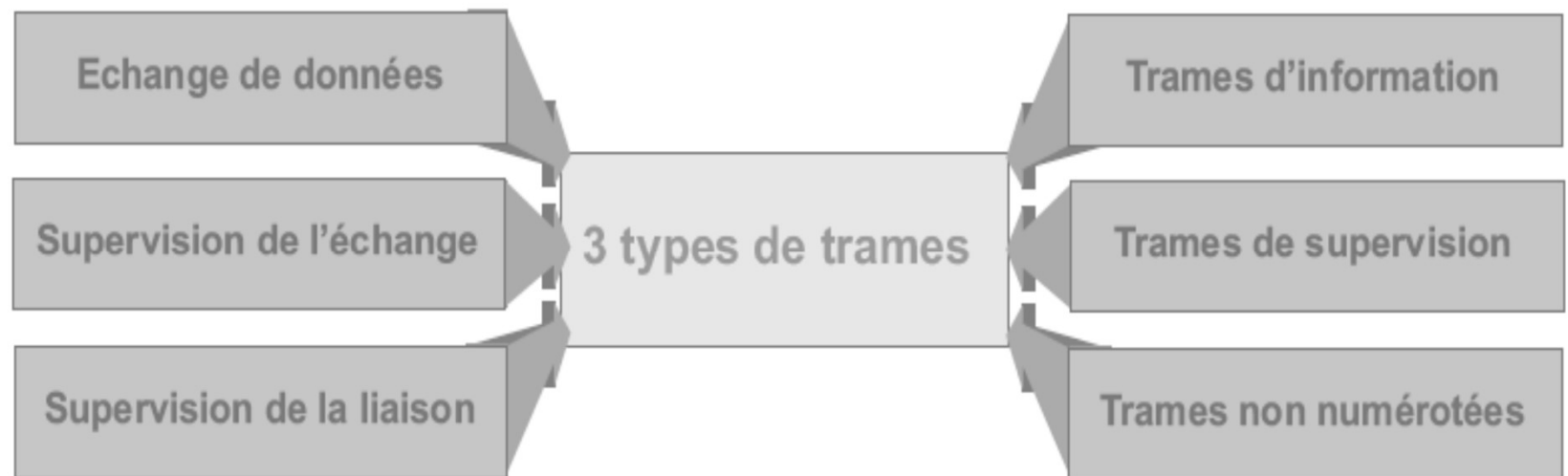
Fonctions

1. Etablissement et rupture de connexion
2. L'échange de données
3. Gestion des temporisateurs
4. Gestion des erreurs
5. Gestion du contrôle du flux

Couche 2 : HDLC

Types de trame HDLC

- On distingue trois types de trames :
 1. Les trames d'**information** ou trames **I** , assurent l'échange des données .
 2. les trames de **supervision** ou trames **S** (Supervisor) , contrôlent le transfert des données (accusé de réception...)
 3. les trames **non numérotées** ou trames **U** (Unnumbered) supervisent la liaison. Les trames **U** sont des trames de signalisation.



Couche 2 : HDLC

Trame

- Chaque trame est délimitée par le ***fanion (Flag)*** qui est aussi employé pour maintenir, en l'absence de données à transmettre, la synchronisation entre les trames.
- Les données varient entre 0 et 2048 octets. Par conséquent, la trame est de longueur variable puisque le champ de données est de longueur variable.
- La trame (sauf les fanions) est transcodée lors de la transmission : toute suite de ***5 bits consécutifs à 1*** est transcodée en une suite de ***cinq bits à 1 et d'un bit à 0***.
Opération inverse au niveau du récepteur.

Couche 2 : HDLC

Trame

- **Abandon** de la transmission d'une trame *au moins sept bits consécutifs à 1*.
- **Remplissage** de l'espace inter-trame par des fanions ou des éléments de *7 à 14 bits consécutifs à 1*.
- Les trames successives peuvent être séparées par un seul fanion.
- Une **liaison est considérée hors service** si une suite *de plus de 15 bits consécutifs à 1* est détectée.

Couche 2 : HDLC

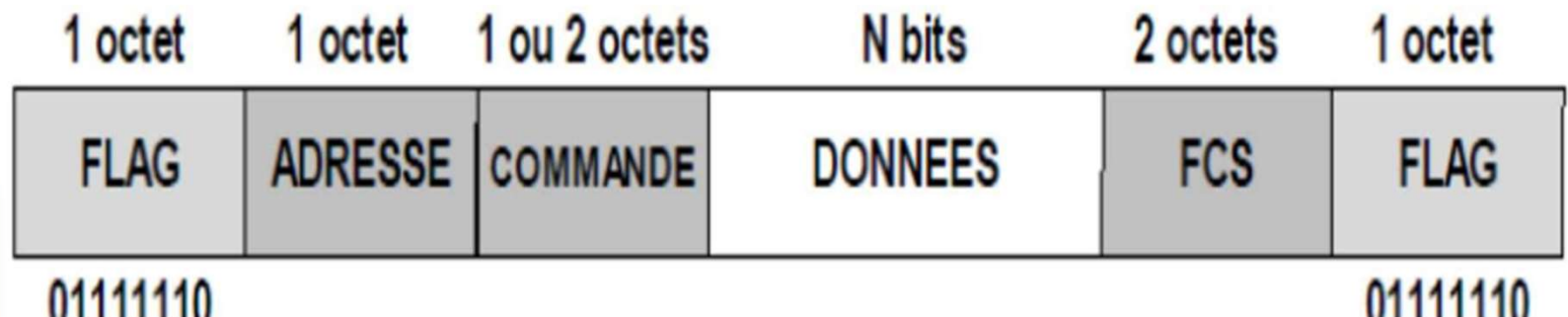
Format de trame

Champs Adresse

- La fonction de ce champ consiste à identifier le sens des émissions des trames de demande et de réponse.
- 2 ADRESSES :**
 - Adresse A → B : 0000 0011 et
 - Adresse B → A : 0000 0001

Champs FCS

- Frame Check Sequence** : est un champ de contrôle d'erreur (CRC) qui consiste à protéger les blocs de données.

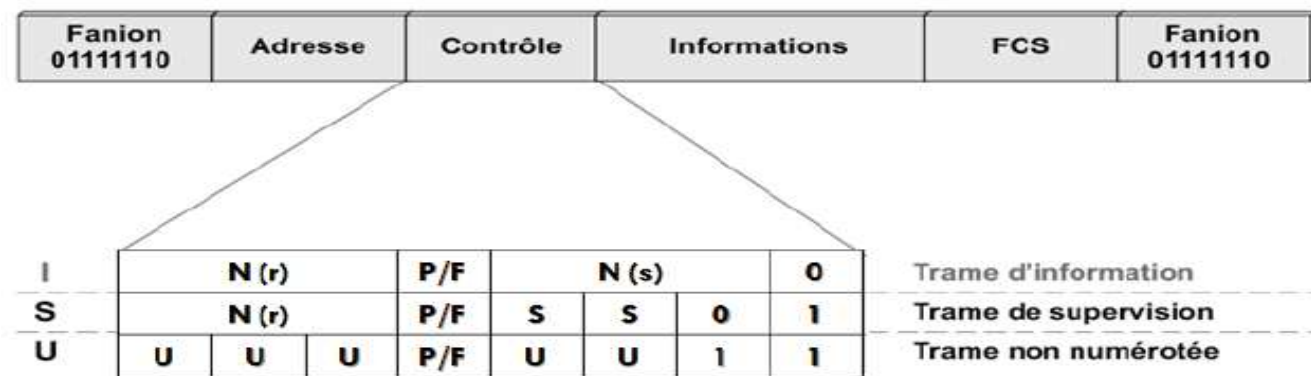


Couche 2 : HDLC

Format générale de la trame HDLC

Champs commande

- **N(S)** et **N(R)** : un compteur de trames émises et reçus (mode standards sur 3 bits ou mode étendu sur 7 *bits*).
- **P/F** (1 *bit*) : **P** positionné à 1 par le primaire lorsque celui-ci sollicite une réponse du secondaire. Le secondaire répond avec **F** = 1 (Final) à la sollicitation du primaire.
- **T** (1 *bit*) : Indique le type de trame



Couche 2 : HDLC

Types trame (T) et fonctions

- **Trames I (Information)** : contiennent un champ de données. avec des compteurs de trames émises N(s) ou reçues N(r). Fonction:
 - ✓ Transportent les données utilisateurs
 - ✓ Acquittement – Retransmission
- **Trames S (Supervision)** : pour superviser l'échange de données. Le champ N(r) permet d'identifier la trame acceptée ou refusée. Fonction :
 - ✓ Acquittement **RR - RNR**
 - ✓ Retransmission **REJ - SREJ**
 - ✓ Contrôle de flux **RR - RNR**
- **Trames U (Unnumbered)** : pour gérer la liaison (établissement, libération...). Elles sont dites non numérotées car elles ne contiennent aucun compteur. Fonctions :
 - **SABM ((Set Asynchronous Balanced Mode)**
 - **UA (Unnumbered Acknowledgement)**
 - **DISC (DISConnect)**

Couche 2 : HDLC

1. Etablissement et rupture de connexion (Trame U)

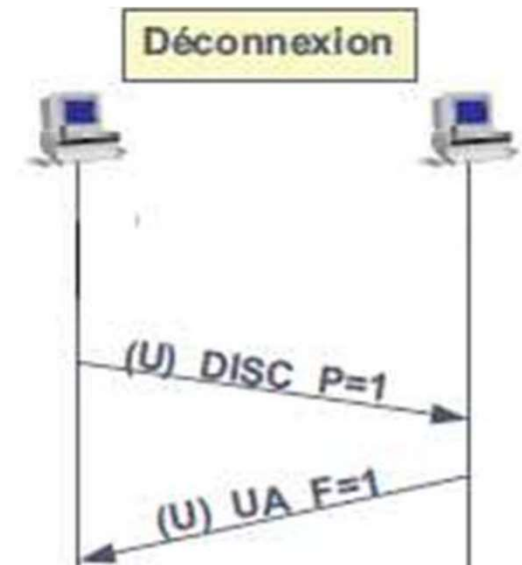
Connexion

- Le primaire demande l'établissement par la trame **SABM (Set Asynchronous Balanced Mode)**, $P = 1$
- Le secondaire, s'il accepte la connexion, répond par la trame **UA (Unnumbered Acknowledgement)**, $F = 1$ identique à celui du bit P .
- La liaison est établie, l'échange d'informations peut donc commencer.



Déconnexion

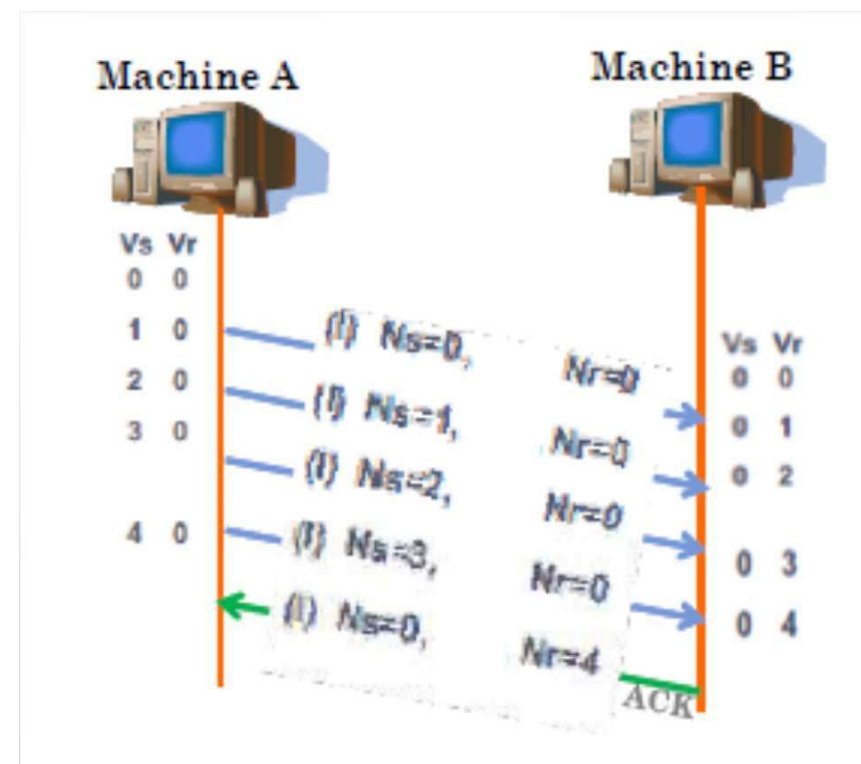
- Le primaire émet une demande de déconnexion **DISC (DISConnect)**.
- Le secondaire accuse la réception avec **UA, $F = P$** de la trame **DISC**
- La liaison est rompue (L'échange de fanions se poursuit pour maintenir la synchronisation tant que la liaison physique n'est pas rompue)



Couche 2 : HDLC

2. Echange de données (Trame I)

- $V(s)$: indique le numéro de la trame à émettre,
- $V(r)$: indique le numéro de la trame attendue
- Après la phase de connexion les compteurs sont initialisés à zéro de chaque côté
- $N(S)$: numéro de la trame émet
- $N(R)$: acquittement des trames reçues de numéro strictement inférieur à $N(S)$



Couche 2 : HDLC

3. Gestion des erreurs (Trame S)

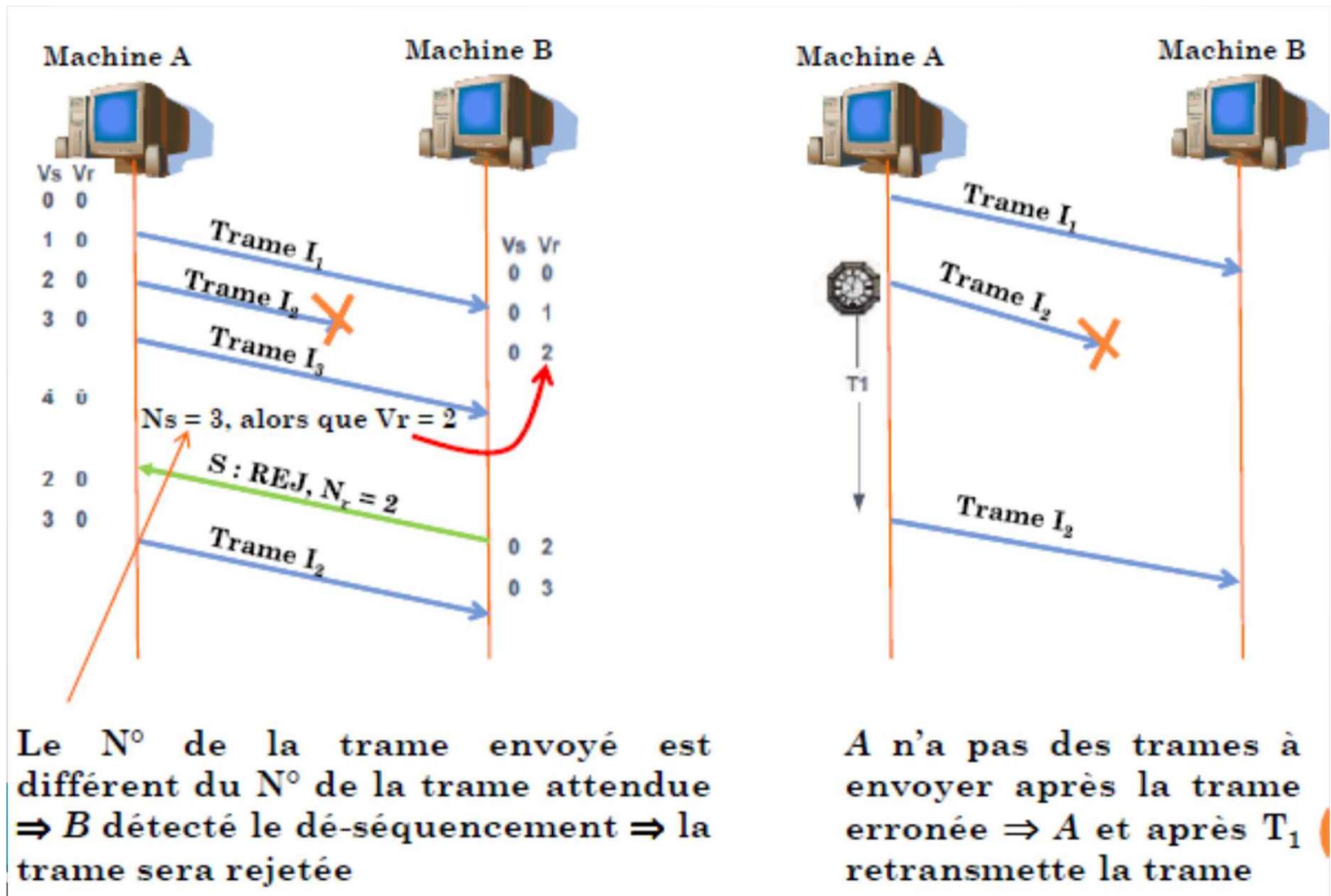
Exemple de la reprise sur erreur

1. Supposons la trame 2 erronée, elle est ignorée par le récepteur.
 - La trame 3 est alors reçue hors séquence, elle est rejetée.
 - La machine *B* émet alors une trame de supervision de rejet (**REJ Reject**) en indiquant à *A* à partir de quelle trame il doit reprendre la transmission [$N(r) = 2$].
 - Toutes les trames dont la valeur de Ns est supérieure à 2 sont alors rejetées (rejet simple).
 - La machine *A* reprend la transmission à partir de la trame 2 ($N(s) = 2$).
2. Supposons que suite à la trame erronée, *A* n'avait plus de données à émettre, alors *B* n'aurait pas détecté le déséquencelement.
 - Dans ce cas, c'est *A* qui à l'échéance du temporisateur **T1**, aurait pris l'initiative de retransmettre la trame 2.

Couche 2 : HDLC

3. Gestion des erreurs (Trame S)

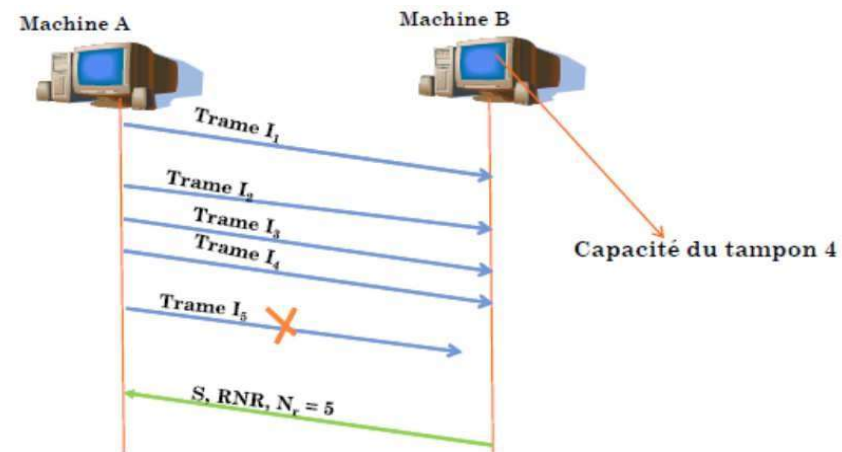
Exemple de la reprise sur erreur



Couche 2 : HDLC

4. Contrôle de flux (Trame S)

- HDLC utilise le contrôle de flux implicite.
- La fenêtre est paramétrée à l'installation du logiciel ou négociée lors de la connexion par le protocole de niveau supérieur.
- En cas de saturation des tampons de réception, le récepteur, (la machine *B*) rejette la trame en excès et informe *A* de son incapacité temporaire à accepter de nouvelles données.
- Machine *B* émet, donc, la trame *S* indique **RNR** (**R**ecieve **N**ot **R**eady) avec le compteur N_r positionné au numéro de la trame reçue et rejetée.



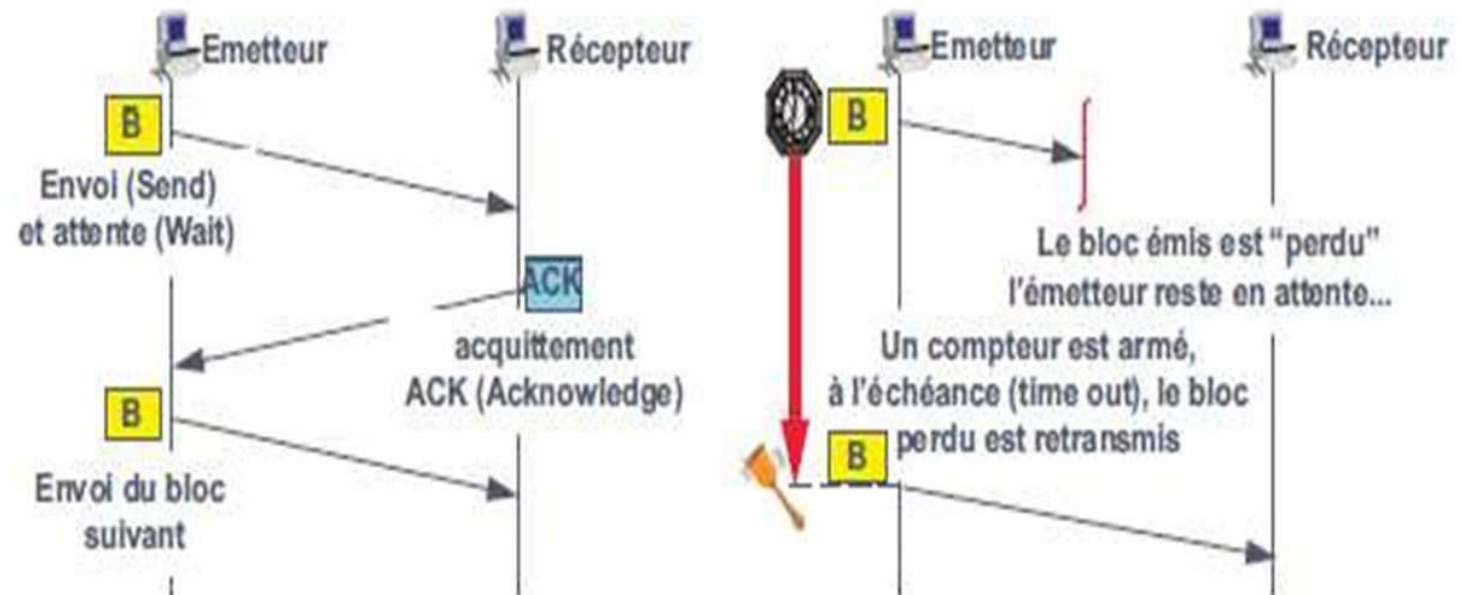
Couche 2 : Contrôle du flux

- Réguler le flux de données entre un émetteur et un récepteur selon la capacité de stockage et de traitement, c-à-d l'émetteur ne doit pas envoyer des trames que si le récepteur est en mesure de les traiter.
- Plusieurs variantes de contrôle de flux, à savoir : protocole de type « envoyer et attendre » (*Send and Wait*) où :
 - ✓ l'émetteur attend des acquittement positifs ou négatifs,
 - ✓ Le récepteur détecte les erreurs, selon le cas, ignore la trame ou demande sa retransmission,

Couche 2 : Protocole Send and wait

Principe de fonctionnement

- ✓ Les données circulent dans un seul sens et une seule trame est envoyée à la fois.
- ✓ Le récepteur informe l'émetteur de son état par un *acquittement*.
- ✓ L'émetteur envoie et attend l'information du récepteur.



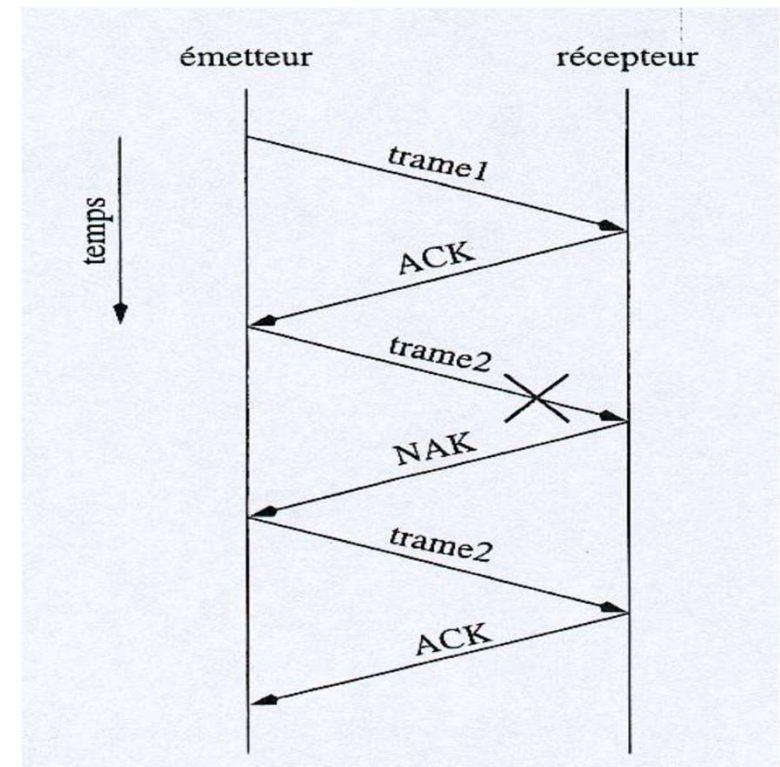
Couche 2 : Protocole Send and wait

Conséquences

- Empêche l'émetteur d'envoyer des données plus rapidement que le récepteur ne peut les traiter.
- Ce protocole sont unidirectionnels, et ne permet qu'une pauvre utilisation de la capacité du canal.

Scénarios

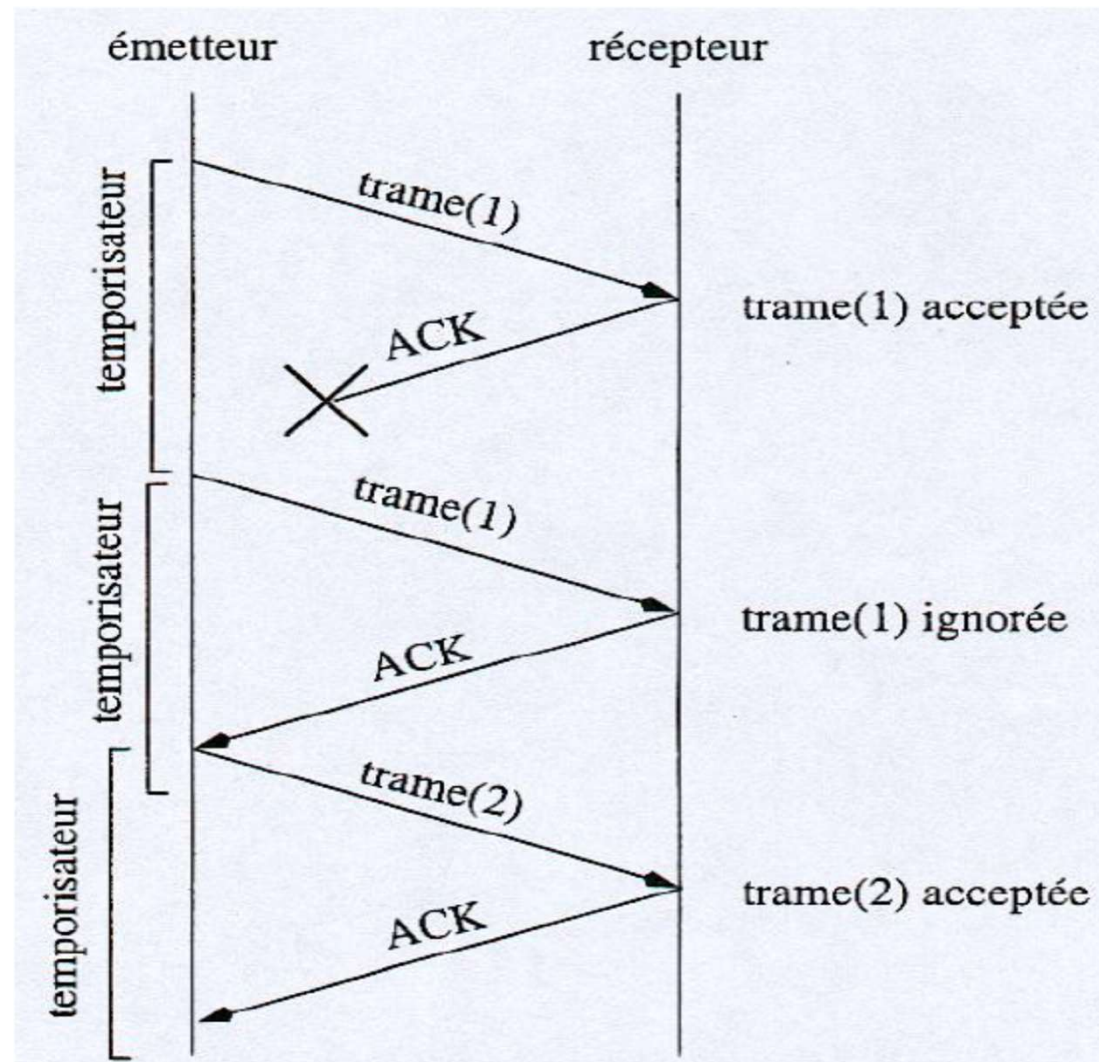
- Acquittement positif ou négatif



Couche 2 : Protocole Send and wait

Scénarios

- Acquittement perdu



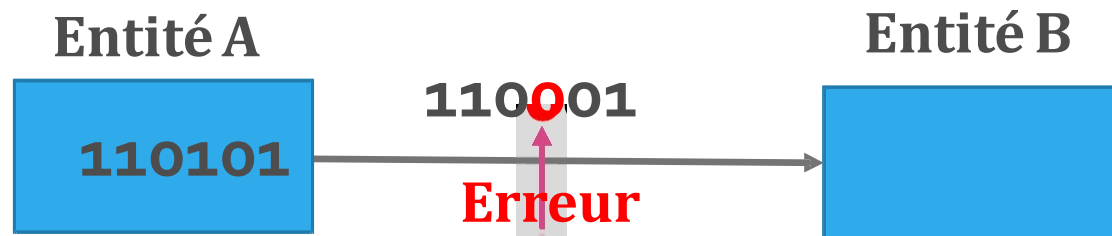
Couche 2 : Contrôle des erreurs

- Nous avons étudié jusque là, tous les mécanismes à mettre en œuvre pour transmettre un flot de bits entre deux systèmes distants
- Cependant, il ne suffit pas d'envoyer seulement l'information, mais il faut encore contrôler l'échange de données
- On appelle protocole, un ensemble de conventions préétablies pour réaliser un échange fiable de données entre deux entités.



Couche 2 : Notion d'erreur

- Lors d'une transmission de données, on doit assurer que les données reçues n'ont pas été altérées durant la transmission (fiabilité de transmission).
- Plusieurs facteurs peuvent modifier le contenu des données.
- Etant donnée, l'extension des réseaux et l'utilisation massive de la fibre optique, **la perte de la synchronisation** des horloges est, aujourd'hui, la principale source d'erreurs.
 - ✓ Comment B peut **détecter** l'occurrence d'une erreur ?
 - ✓ Comment B peut **localiser** une erreur ?
 - ✓ Comment B peut **corriger** une erreur ?



Couche 2 : Notion d'erreur

Taux d'Erreur Binaire BER

- On appelle taux d'erreur binaire ou **BER (Bit Error Rate)** le rapport entre le nombre d'informations (bits) erronées reçues et le nombre d'informations (bits) transmises

$$\text{BER} = \text{Nb_bits_Erronnés} / \text{Nb_Total_bits}$$

Exemple

- La suite transmise est : «011001001100100101001010 »
- La suite reçue est : «011001101100101101000010 »
- Nombre de bites transmise est : 24
- Nombre de bites erroné est : 3
 - ✓ Donc, $\text{BER} = 3/24 = 0,125$

Couche 2 : Notion d'erreur

Taux d'Erreur Binaire BER

- Le taux d'erreur binaire varie en pratique de 10^{-4} à 10^{-9} (réseaux locaux)
- Dans les réseaux, les erreurs se produisent généralement par rafale. L'erreur affecte aléatoirement **n bits** consécutifs et **non 1 bit tous les x bits**
- Si T_e est la probabilité pour qu'un bit soit erroné, la probabilité de recevoir un bit correct est de $(1 - T_e)$

La probabilité de recevoir un bloc sans erreur est d'autant plus faible que la longueur du bloc est grande

Couche 2 : Détection d'erreur

- La détection d'erreur est les mécanismes mis en œuvre pour que le système destinataire puisse vérifier la validité des données reçues
- La détection d'erreur repose sur l'introduction d'une certaine redondance dans l'information transmise
- Quatre techniques peuvent être mises en œuvre pour détecter et éventuellement corriger les erreurs :
 1. La détection par écho
 2. La détection par répétition
 3. La détection par clé calculée
 4. La détection et correction d'erreur par code

Couche 2 : Détection d'erreur

1. Détection par écho

Principe

- Le récepteur renvoie en écho le message reçu à l'émetteur.
- Si le message est différent de celui émis, l'émetteur retransmet le message.

Utilisation

- Cette technique est utilisée dans les terminaux asynchrones.

2. Détection par répétition

- Chaque message émis est suivi de sa réplique.
- Si les deux messages sont différents, le récepteur demande une retransmission.

Utilisation

- Cette technique est utilisée dans les milieux sécurisés très perturbés et dans certaines applications dites temps réel.

Couche 2 : Détection d'erreur

Discussion

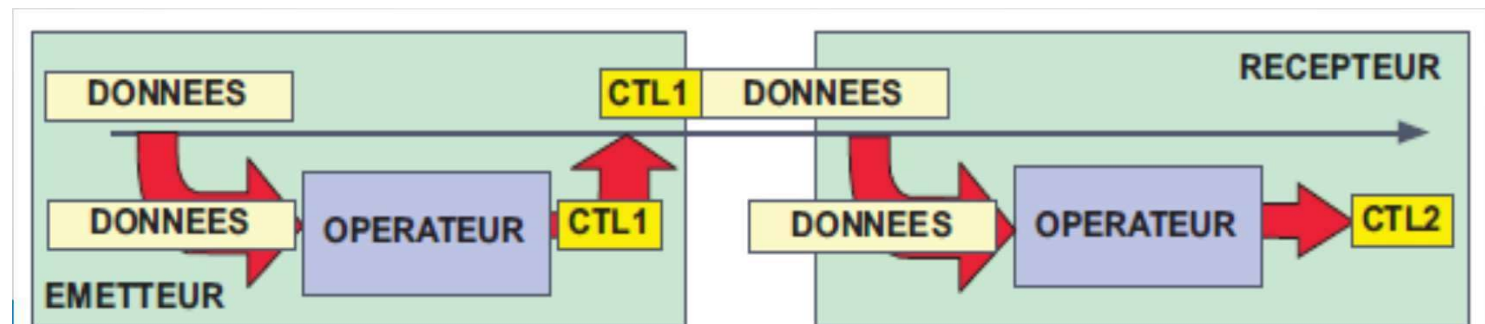
- La détection et la correction des erreurs nécessitent d'introduire de la **redondance** dans les messages transmis.
- Certaines erreurs ne peuvent pas être détectées
 - ✓ Exemple : la même erreur sur les deux exemplaires
- Certaines erreurs détectées ne peuvent pas être corrigées
 - ✓ Exemple : une erreur différente sur au moins deux exemplaires.
- Certaines erreurs sont mal corrigées
 - ✓ Exemple : une même erreur sur deux exemplaires simultanément.

Couche 2 : Détection d'erreur

3. Détection par clé calculée

Principe

- Une information supplémentaire (clé) déduite des informations transmises est ajoutée à celles-ci.
- L'émetteur calcule une séquence de contrôle **CTL₁** par une opération mathématique appliquée au message à émettre et envoyée avec le message.
- Le récepteur recalcule la clé **CTL₂** en utilisant la même opération.
 - ✓ si **CTL₁** identique à **CTL₂**, alors le bloc est exact,
 - ✓ sinon le bloc est rejeté.



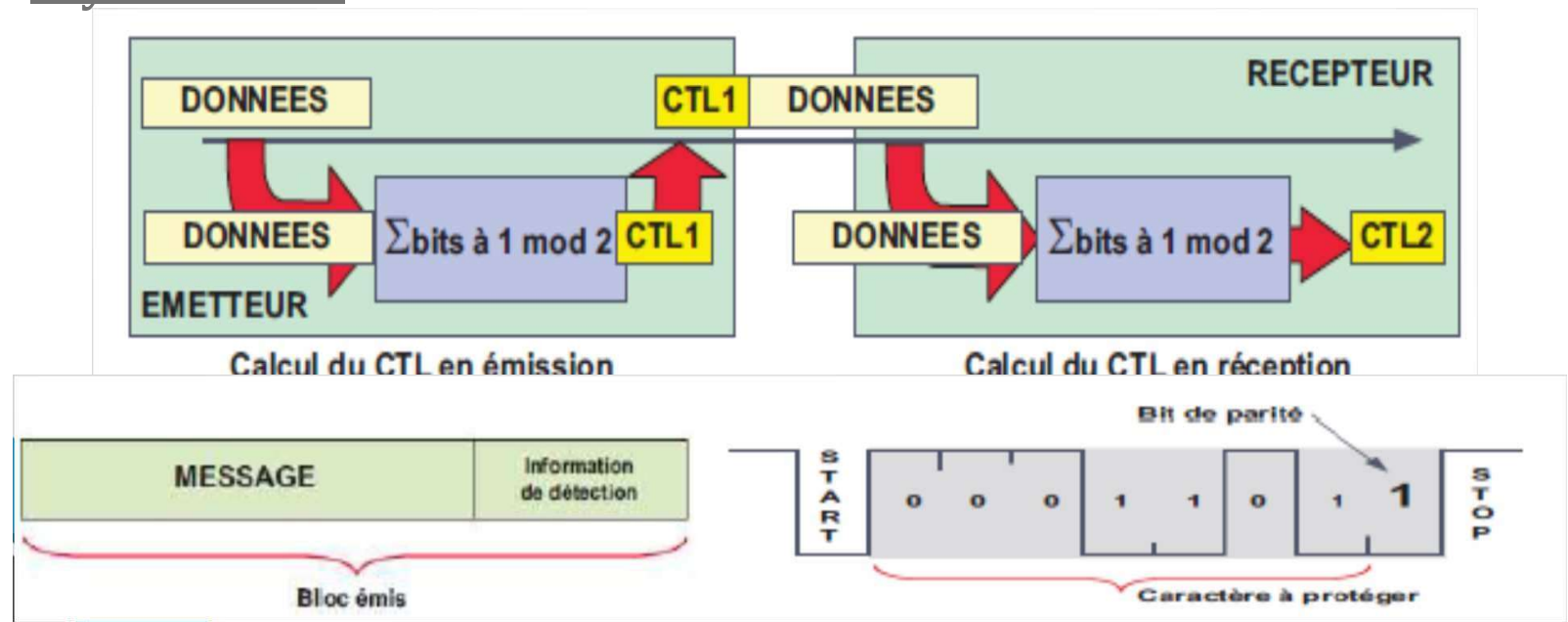
Couche 2 : Détection d'erreur

3. Détection par clé calculée

A – Bit de parité VRC (Vertical Redundancy Check)

Principe : $\sum \text{bits} \text{ à } 1 \bmod 2$ (la parité peut être paire ou impaire)

- Vérification par redondance verticale ne permet de détecter que les erreurs portant sur un nombre impair de bits.
- Elle est, essentiellement, utilisée dans les transmissions asynchrones.



Couche 2 : Détection d'erreur

3. Détection par clé calculée

A – Bit de parité VRC (Vertical Redundancy Check)

Exemple

Lettre	Code ASCII	(parité paire)	(parité impaire)
G	1110001	11100010	11100011
T	0100001	01000010	01000011
R	0100101	01001011	01001010

Couche 2 : Détection d'erreur

3. Détection par clé calculée

B – Bit de parité LRC (Longitudinal Redundancy Check)

Principe : $\sum$ bits à 1 mod 2 de chaque bit + $\sum$ bits à 1 mod 2 pour tous : le 1ier bit, 2ème bit, ...le **LRC** est lui-même protégé par un bit de parité **VRC**. Dans les transmissions synchrones, les caractères sont envoyés en blocs.

Exemple

B	j	n	o	r	u
42	6A	6E	6F	72	75

Lettre	ASCII	Binaire	VRC pair
B	42	1000010	0
o	6F	1101111	0
n	6E	1101110	1
j	6A	1101010	0
o	6F	1101111	0
u	75	1110101	1
r	72	1110010	0
LRC impair		0111110	

VRC pair	⇒	1000010 <u>0</u> 1101111 <u>0</u> 1101110 <u>1</u> 1101010 <u>0</u> 1101111 <u>0</u> 1110101 <u>1</u> 1110010 <u>0</u>
LRC impair	⇒	1000010 1101111 1101110 1101010 1101111 1110101 1110010 <u>0111110</u>

Couche 2 : Détection d'erreur

3. Détection par clé calculée

C – Code CRC (Contrôle de Redondance Cyclique)

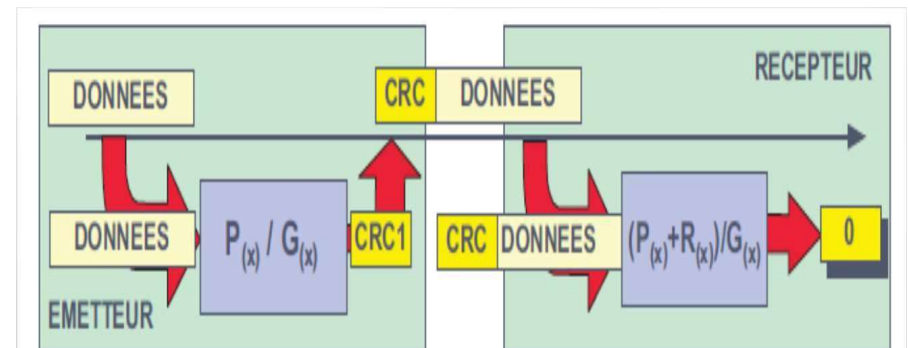
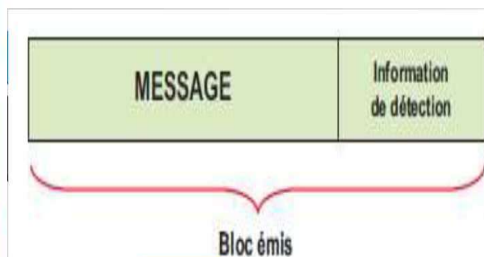
parfois appelé aussi FCS (Frame Check Sequence)

Principe :

- Considère le bloc de N bits à transmettre comme un polynôme de degré $N - 1 : P(x)$
- Ce polynôme est divisé par un autre, dit polynôme générateur $G(x)$, selon les règles de l'arithmétique booléenne ou arithmétique modulo 2.
- Le reste de cette division $R(x)$ constitue le **CRC**

Emetteur : $T(x) = P(x) \cdot x^k + R(x)$ où k : le plus grand degré de $G(x)$

Récepteur : si le reste de $T(x)/G(x)$ égale à 0, alors pas d'erreur



Couche 2 : Détection d'erreur

3. Détection par clé calculée

C – Code CRC (Contrôle de Redondance Cyclique)

parfois appelé aussi FCS (Frame Check Sequence)

Exemple :

- On désire protéger le message « 110111 » par une clé calculée à l'aide du polynôme générateur : $x^2 + x + 1$

Réponse :

- Au message 110111, on fait correspondre le polynôme : $x^5 + x^4 + 0x^3 + x^2 + x^1 + x^0$

Donc :

$$(x^5 + x^4 + 0x^3 + x^2 + x^1 + 1) \cdot x^2 = x^7 + x^6 + 0x^5 + x^4 + x^3 + x^2 + 0 + 0$$

$$T(x) = x^7 + x^6 + 0x^5 + x^4 + x^3 + x^2 + x + 1$$

Message transmis:
011011111

$x^7 + x^6 + 0 + x^4 + x^3 + x^2 + 0 + 0$	$x^2 + x + 1$
$ \begin{array}{r} x^7 \quad x^6 \quad x^5 \quad \downarrow \quad \downarrow \quad \downarrow \\ x^5 \quad x^4 \quad x^3 \quad \downarrow \\ x^5 \quad x^4 \quad x^3 \quad \downarrow \\ x^2 \quad 0 \quad 0 \\ x^2 \quad x \quad 1 \\ \text{RESTE} \Rightarrow x \quad 1 \end{array} $	$ \begin{array}{r} x^5 \quad x^3 \quad 1 \end{array} $

Exercices

1. Avec la protection de contrôle parité paire, le récepteur reçoit le mot suivant : 01011011. Que peut on déduire?
2. Soit le message composé de la chaîne « NET », le contrôle de transmission de chaque caractère est assuré par le bit de parité. Donnez la représentation binaire du message transmis (supposant que les caractères sont codés en ASCII sur 7 bits)
3. Soit le message 0011111101, on rajoute à ce message un CRC calculé par le polynôme générateur : $x^2 + x + 1$
- Quel est le message codé?
4. On considère que le polynôme générateur $x^3 + x^2 + 1$ et le message 1001
 - ✓ Calculez les bits de contrôle
 - ✓ Quel est le message à envoyer?
 - ✓ Comment peut-on vérifier l'intégrité du message reçu?